

Requirements Gathering – Banking Transaction System

Software tester course project

1. Stakeholder Analysis

Stakeholder	Role	Main interest	Influence	Interest
Customer	End user	Fast, safe transfers, deposits, withdrawals, balance checks	Medium	High
Bank Teller / Employee	Internal user	Process and look up transactions accurately	Medium	High
Bank Manager	Decision maker	Reports, audit trail, fraud control	High	High
System Administrator	Technical operator	User accounts, roles, backups, uptime	High	Medium
Auditor / Compliance Officer	Regulator-facing	Complete, tamper-proof transaction logs	High	Medium
Development Team	Builders	Clear, stable requirements	High	High

QA / Testers	Verifiers	Testable, unambiguous requirements	Medium	High
Course Instructor	Evaluator	Quality of process and deliverables	High	Medium

Power/interest strategy

- Manage closely (high influence, high interest): Bank Manager, Development Team, Instructor.
- Keep satisfied (high influence, lower interest): System Administrator, Auditor.
- Keep informed (lower influence, high interest): Customer, Teller, QA.

Elicitation techniques

interviews with a teller or manager, a short customer questionnaire, studying existing banking apps, and a team workshop to prioritize with MoSCoW (Must / Should / Could / Won't).

2. User Stories

s.

Customer

- **US-01 Login:** As a customer, I want to log in with a username and password, so that only I can access my account. AC: valid credentials open the dashboard; invalid credentials show an error; the account locks after 3 failed attempts.
- **US-02 Balance:** As a customer, I want to view my current balance, so that I know how much I can spend. AC: the balance matches the database and updates after each transaction.
- **US-03 Deposit:** As a customer, I want to deposit money, so that my balance increases. AC: the amount must be greater than 0; a record is saved with date and time.
- **US-04 Withdraw:** As a customer, I want to withdraw money, so that I can access my cash. AC: rejected if the amount exceeds the balance; the balance never goes negative.
- **US-05 Transfer:** As a customer, I want to transfer money to another account, so that I can pay people. AC: both accounts update atomically; if one step fails, neither changes; an invalid recipient is rejected.
- **US-06 History:** As a customer, I want to see my transaction history, so that I can track my spending. AC: transactions are listed newest first and can be filtered by date.

Teller

- **US-07 Search:** As a teller, I want to search for an account by number, so that I can help customers quickly.
- **US-08 Assist:** As a teller, I want to perform deposits and withdrawals on a customer's behalf, so that in-branch service is possible.

Manager / Admin / Auditor

- **US-09 Reports:** As a manager, I want daily transaction reports, so that I can monitor activity.

- **US-10 Users:** As an admin, I want to create, disable, and assign roles to users, so that access is controlled.
- **US-11 Audit:** As an auditor, I want an immutable log of all transactions, so that I can verify compliance.
- **US-12 Fraud flag:** As a manager, I want transactions above a set limit flagged, so that suspicious activity is reviewed.

3. Functional Requirements

ID	Requirement	Priority	Story
FR-01	The system shall authenticate users by username and password.	Must	US-01
FR-02	The system shall lock an account after 3 consecutive failed logins.	Must	US-01
FR-03	The system shall display the current account balance.	Must	US-02
FR-04	The system shall allow deposits of amounts greater than zero.	Must	US-03
FR-05	The system shall allow withdrawals only when the balance is sufficient.	Must	US-04
FR-06	The system shall transfer funds between two	Must	US-05

	valid accounts as one atomic operation.		
FR-07	The system shall record every transaction with ID, type, amount, accounts, and timestamp.	Must	US-06, US-11
FR-08	The system shall display transaction history with date filtering.	Should	US-06
FR-09	The system shall support role-based access (Customer, Teller, Manager, Admin).	Must	US-10
FR-10	The system shall let tellers search accounts by account number.	Should	US-07
FR-11	The system shall generate daily transaction reports.	Should	US-09
FR-12	The system shall flag transactions above a configurable	Could	US-12

	limit.		
FR-13	The system shall expose REST endpoints for transactions so they can be tested with Postman.	Should	All

4. Non-Functional Requirements

ID	Category	Requirement	How to verify
NFR-01	Performance	Any transaction completes in under 2 seconds with up to 50 concurrent users.	Postman runner / load test
NFR-02	Security	Passwords are stored hashed (e.g., BCrypt), never in plain text.	Database inspection
NFR-03	Security	All inputs are validated to prevent SQL injection and XSS.	Security test cases
NFR-04	Security	Sessions expire after 10 minutes of inactivity.	Selenium test
NFR-05	Reliability	Transactions are ACID-compliant, with no partial updates.	Failure-injection tests
NFR-06	Availability	The system is	Monitoring

		available 99% of the time during working hours.	
NFR-07	Usability	A new user completes a transfer in under 3 minutes without help.	Usability test
NFR-08	Maintainability	Code follows Java conventions and is versioned on GitHub, with tasks tracked in Jira.	Code review
NFR-09	Compatibility	The web UI works on the latest Chrome and Firefox.	Selenium cross-browser run
NFR-10	Auditability	Transaction logs cannot be edited or deleted by any role.	Access-control tests
NFR-11	Data integrity	Monetary values are stored with exact precision (DECIMAL, not float).	Schema review

Tester-track tip

Tip for the tester track: every FR and NFR should map to at least one test case. Keep a traceability matrix (Requirement ID → User Story → Test Case ID) in Jira or a spreadsheet.